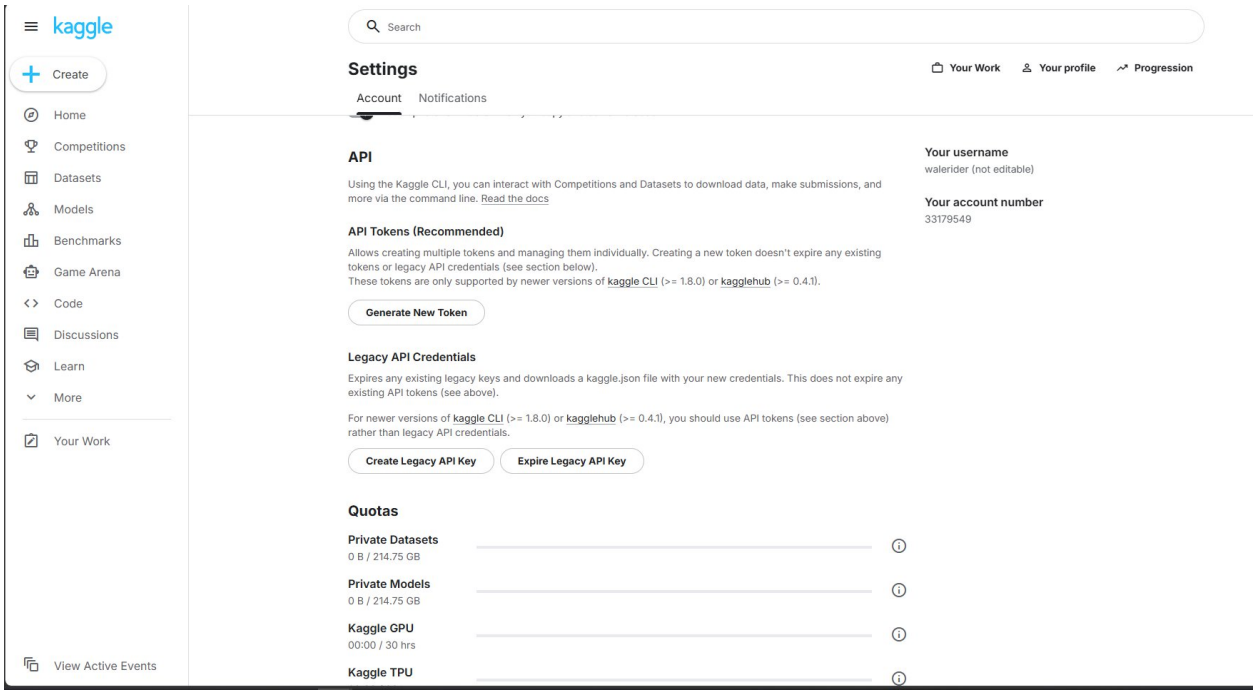
Математический анализ и python

Язык программирования Python в последнее время все чаще используется для анализа данных, как в науке, так и коммерческой сфере. Этому способствует простота языка, а также большое разнообразие открытых библиотек.

Сегодня мы разберем простой пример исследования и классификации данных с использованием некоторых библиотек на Python. Для исследования, нам понадобится выбрать интересующий нас набор данных (DataSet). Разнообразные наборы Dataset'ы можно скачать с **сайта** (нам скачивать не обязательно). Подберите себе подходящий dataset. Далее вам понадобиться там зарегистрироваться. Лично я зашёл через google аккаунт.

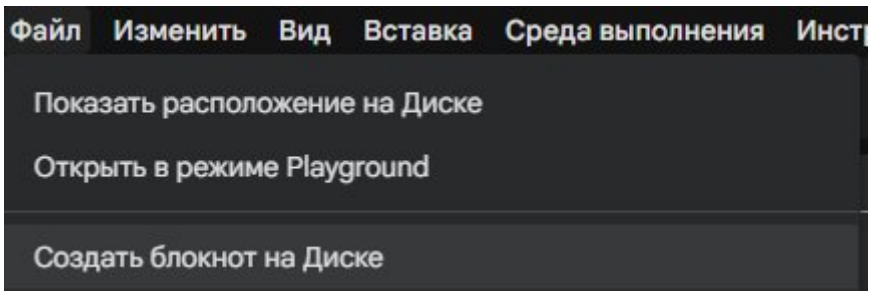
После успешной регистрации вам надо найти раздел Settings, и там API:



Сгенерируйте себе API токен, он нам в дальнейшем понадобится. **Сохраните токен**.

Далее, перейдём на google colab. Это бесплатный облачный сервис для программирования на Python. Тут вам придётся залогиниться в свой аккаунт.

Создайте новый блокнот. Для этого, найдите файл -> создать блокнот на диске.



И пропишите следующий код:

```
!pip install kagglehub

!pip install matplotlib

!pip install pandas

!pip install numpy
```

Он позволит нам установить основные библиотеки в нашу среду. Обратите внимание, что при использовании pip перед названием надо писать восклицательный знак.

Далее нам понадобятся следующие модули:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
import os

from matplotlib import cm

from mpl_toolkits.mplot3d import Axes3D

from kagglehub import KaggleDatasetAdapter

import kagglehub
```

Для того, что бы улучшить наглядность графиков, напишем вспомогательную функцию для генерации цветовой схемы. На входе функция принимает количество цветов, которое необходимо сгенерировать. Функция возвращает связный список с цветами.

```
PLOT_LABEL_FONT_SIZE = 14

def getColors(n):

    COLORS = []

    cm = plt.cm.get_cmap('hsv', n)

    for i in np.arange(n):

        COLORS.append(cm(i))

    return COLORS
```

Напишем еще одну вспомогательную функцию для сортировки объектов dict.

dict — представляет собой встроенный тип Python, где данные хранятся в виде пары ключ-значения. Функция сортирует словарь по значениям в убывающем порядке и возвращает отсортированные список ключей и

соответсвуюущий ему по порядку следования элементов список значений. Эта функция будет полезна при построении гистограмм.

```
def dict_sort(my_dict):
    keys = []
    values = []
    my_dict = sorted(my_dict.items(), key=lambda x:x[1], reverse=True)
    for k, v in my_dict:
        keys.append(k)
        values.append(v)
    return (keys,values)
```

Мы добрались до непосредственно данных.

Первое, что мы должны сделать, это разделить наш код на несколько частей. В google colab можно делить код, для того, чтобы отделять части, которые нужно запустить только 1 раз и части, которые постоянно будут меняться.Получится примерно так:

```
!pip install kagglehub
!pip install matplotlib
!pip install pandas
!pip install numpy
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: kaggle-sdk<1.0,>=0.1.14 in /usr/local/lib/python3.12/dist-packages (from kagglehub) (0.1.16)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (26.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.14->kagglehub) (5.29.6)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2026.2.25)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_13161/1528313246.py in <cell line: 0>()
     13 kgat_token = 'KGAT_f4bcffa6377c30ec2dae34617edd68cd'
     14 os.environ['KAGGLE_API_TOKEN'] = kgat_token
---> 15 kagglehub.login()
     16 PLOT_LABEL_FONT_SIZE = 14
     17 def getColors(n):

NameError: name 'kagglehub' is not defined
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from matplotlib import cm
from mpl_toolkits.mplot3d import Axes3D
from kagglehub import KaggleDatasetAdapter
import kagglehub
```

```
kagglehub.login()
```

Kaggle credentials successfully validated.

```
Kaggle credentials set.
Kaggle credentials successfully validated.
```

```
kgat_token = 'KGAT_f4bcffa6377c30ec2dae34617edd68cd'
os.environ['KAGGLE_API_TOKEN'] = kgat_token
PLOT_LABEL_FONT_SIZE = 14
def getColors(n):
    COLORS = []
    cm = plt.cm.get_cmap('hsv', n)
    for i in np.arange(n):
        COLORS.append(cm(i))
    return COLORS
def dict_sort(my_dict):
    keys = []
    values = []
    my_dict = sorted(my_dict.items(), key=lambda x:x[1], reverse=True)
    for k, v in my_dict:
        keys.append(k)
        values.append(v)
    return (keys,values)
data_path = kagglehub.dataset_download("rhythmghai/food-ordering-behavior-india-50k-orders")
file_path = os.path.join(data_path, " food_ordering_behavior_dataset.csv")

df = kagglehub.dataset_load(
  KaggleDatasetAdapter.PANDAS,
  "rhythmghai/food-ordering-behavior-india-50k-orders",
  "food_ordering_behavior_dataset.csv",
)
```

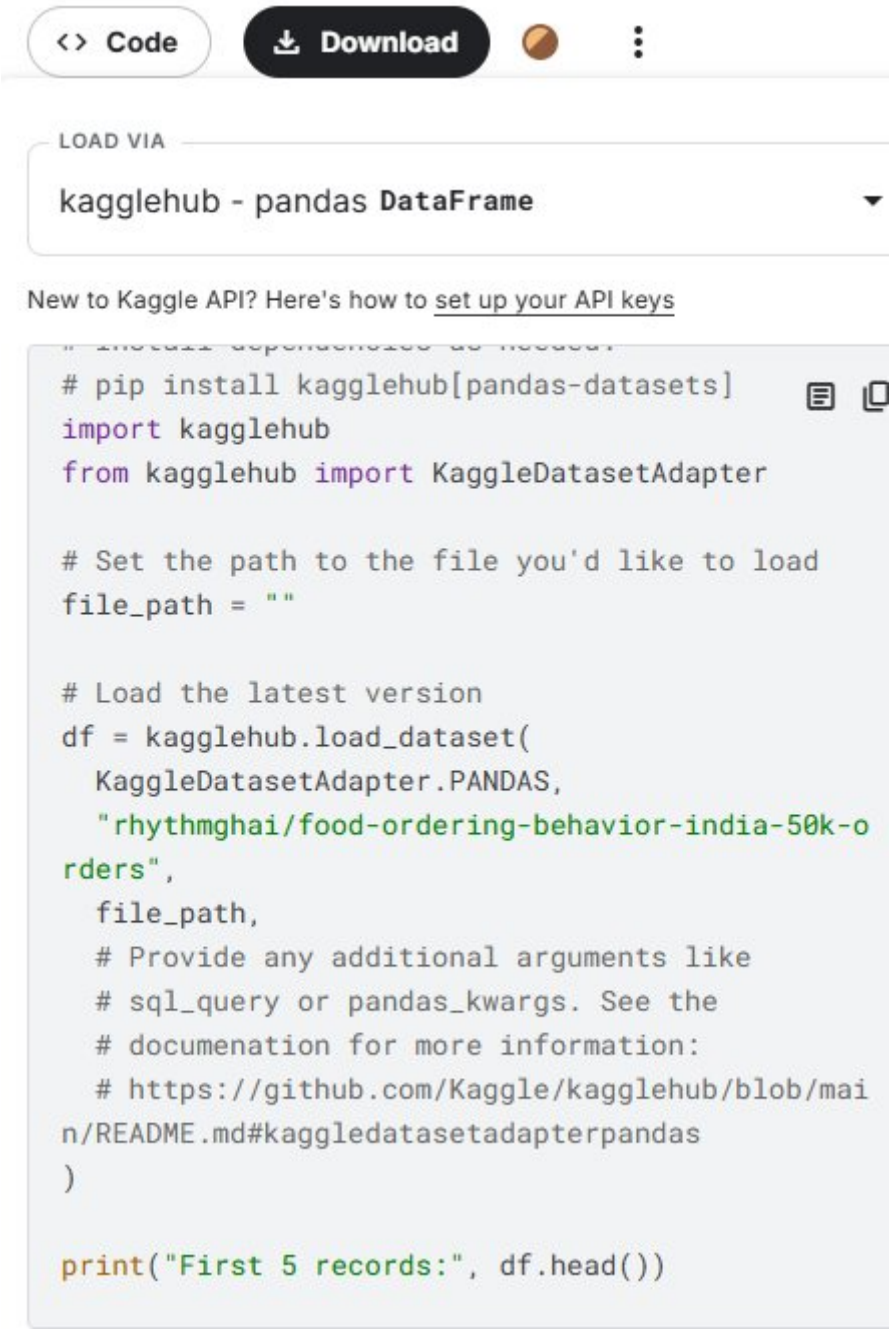
```
Using Colab cache for faster access to the 'food-ordering-behavior-india-50k-orders' dataset.
Using Colab cache for faster access to the 'food-ordering-behavior-india-50k-orders' dataset.
```

После импортирования модулей пропишите:

```
kagglehub.login()
```

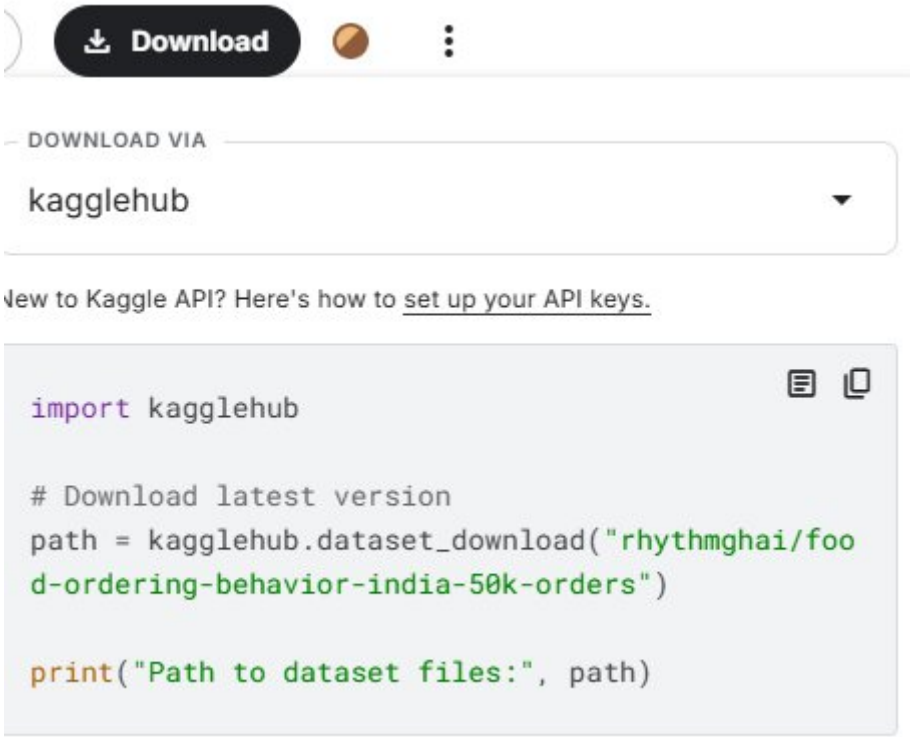
И вставьте туда ваш api ключ.

Далее в коде вам нужно будет создать переменную df. Для неё вам надо перейти к выбранному вами dataset’у и нажать <>code



И скопировать строчку с df.

Далее, нажимаете кнопку download и копируете тот код, который вам поможет установить dataset



После этого, соединяете оба этих кода. Получится примерно следующее:

```
data_path = kagglehub.dataset_download("rhythmghai/food-ordering-behavior-india-50k-orders")

file_path = "food_ordering_behavior_dataset.csv"

df = kagglehub.dataset_load(

  KaggleDatasetAdapter.PANDAS,

  "rhythmghai/food-ordering-behavior-india-50k-orders",

  file_path,

)
```

Где вместо file_path вы прописываете себе название .csv файла с сайта.

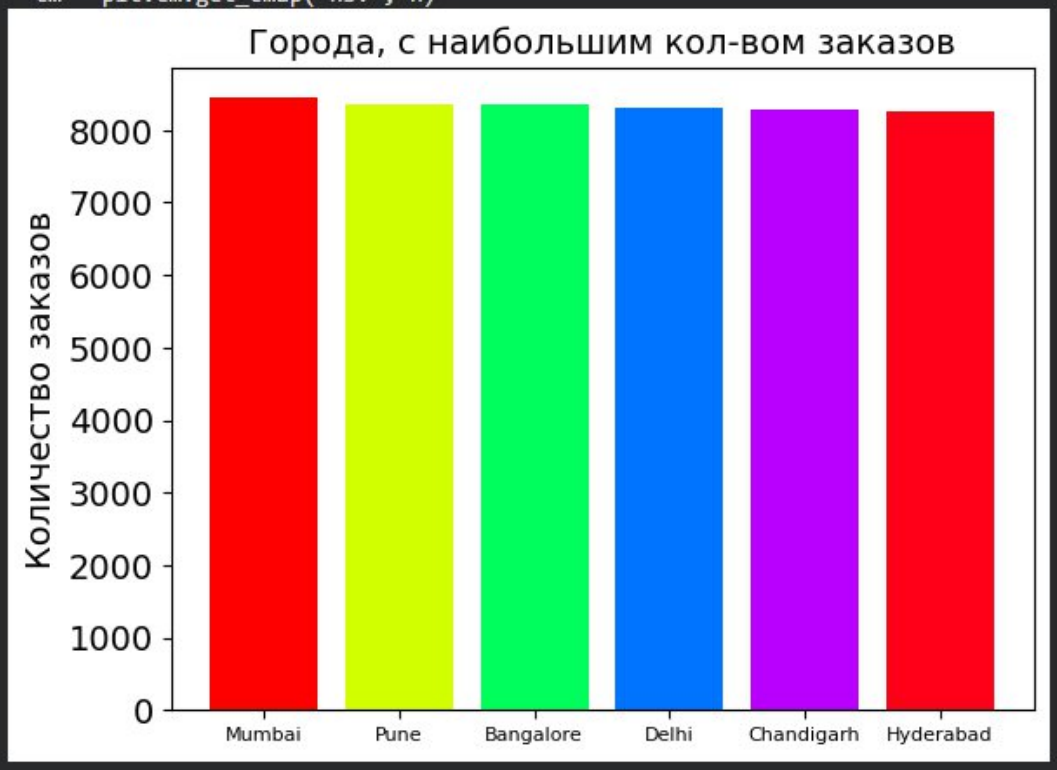
Первичную обработку данных на этом завершаем.

Построим график наблюдений по странам. Для построения графиков используется библиотека pyplot. Примеры построения простого графика можно найти на официальном сайте https://matplotlib.org/users/pyplot_tutorial.html. Для построения гистограммы можно использовать метод bar. В моём случае я буду смотреть города.

Получается следующее:

```
cities_count = pd.value_counts(df['city'].values, sort=True)
cities_count_keys, cities_count_values = dict_sort(dict(cities_count))
TOP_CITY = len(cities_count_keys)
plt.title('Города, с наибольшим кол-вом заказов', fontsize=PLOT_LABEL_FONT_SIZE)
plt.bar(np.arange(TOP_CITY), cities_count_values, color=getColors(TOP_CITY))
plt.xticks(np.arange(TOP_CITY), cities_count_keys, rotation=0, fontsize=8)
plt.yticks(fontsize=PLOT_LABEL_FONT_SIZE)
plt.ylabel('Количество заказов', fontsize=PLOT_LABEL_FONT_SIZE)
plt.show()
```

```
/tmp/ipykernel_13161/1603731443.py:1: FutureWarning: pandas.value_counts is deprecated and w
  cities_count = pd.value_counts(df['city'].values, sort=True)
/tmp/ipykernel_13161/1865259703.py:6: MatplotlibDeprecationWarning: The get_cmap function wa
  cm = plt.cm.get_cmap('hsv', n)
```



Далее, мне, к примеру, интересно сделать круговую диаграмму возрастов посетителей. Для этого можно использовать функцию .pie()

```
age_groups = {
    '18-25': (18, 25),
    '26-30': (26, 30),
    '31-35': (31, 35),
    '36-40': (36, 40),
    '41-45': (41, 45)
}
counts = {}
for group, (min_age, max_age) in age_groups.items():
    counts[group] = df[(df['age'] >= min_age) & (df['age'] <= max_age)].shape[0]
labels = list(counts.keys())
sizes = list(counts.values())
plt.figure(figsize=(10, 8))
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90,
        colors=plt.cm.Set3(np.linspace(0, 1, len(labels))))
plt.title('Распределение возрастов клиентов', fontsize=16, pad=20)
plt.axis('equal')
plt.show()
```

Задание:

Выбрать 4 интересующих вас колонки и сделать для них:

Столбчатую диаграмму, круговую диаграмму, гистограмму и диаграмму рассеяния